STEM BUILDERS

Variables, Strings & Numbers

Variables, Strings, and Numbers

In this section, you will learn to store information in variables. You will learn about two types of data: strings, which are sets of characters, and numerical data types.

Variables

A variable holds a value.

Example

```
message = "Hello Python world!"
print(message)
```

Output:

```
Hello Python world!
```

A variable holds a value. You can change the value of a variable at any point.

```
message = "Hello Python world!"
print(message)
message = "Python is my favorite language!"
print(message)
```

Output:

```
Hello Python world! Python is my favorite language!
```

Naming rules

- Variables can only contain letters, numbers, and underscores. Variable names can start with a letter or an underscore, but can not start with a number.
- Spaces are not allowed in variable names, so we use underscores instead of spaces. For example, use student_name instead of "student name".
- You cannot use Python keywords as variable names.
- Variable names should be descriptive, without being too long. For example mc_wheels is better than just "wheels", and number_of_wheels_on_a_motorycle.
- Be careful about using the lowercase letter l and the uppercase letter O in places where they could be confused with the numbers 1 and 0.

NameError

There is one common error when using variables, that you will almost certainly encounter at some point. Take a look at this code, and see if you can figure out why it causes an error.

```
message = "Thank you for sharing Python with the world, Guido!"
print(mesage)
```

Output:

```
--------------------------------------------------------------------------- NameError
Traceback (most recent call last)
/home/ehmatthes/development_resources/project_notes/intro_programming/notebooks/<ipython-
input-12-7966723379c3> in <module>()       1 message = "Thank you for sharing Python with
the world, Guido!" ----> 2 print(mesage)  NameError: name 'mesage' is not defined
```

Let's look through this error message. First, we see it is a NameError. Then we see the file that caused the error, and a green arrow shows us what line in that file caused the error. Then we get some more specific feedback, that "name 'mesage' is not defined".

You may have already spotted the source of the error. We spelled message two different ways. Python does not care whether we use the variable name "message" or "mesage". Python only cares that the spellings of our variable names match every time we use them.

This is pretty important, because it allows us to have a variable "name" with a single name in it, and then another variable "names" with a bunch of names in it.

We can fix NameErrors by making sure all of our variable names are spelled consistently.

```
message = "Thank you for sharing Python with the world, Guido!"
print(message)
```

Output:

```
Thank you for sharing Python with the world, Guido!
```

In case you didn't know Guido van Rossum created the Python language over 20 years ago, and he is considered Python's Benevolent Dictator for Life. Guido still signs off on all major changes to the core Python language.

Exercises

Hello World - variable

- Store your own version of the message "Hello World" in a variable, and **print** it.

One Variable, Two Messages:

- Store a message in a variable, and then **print** that message.
- Store a new message in the same variable, and then **print** that new message.

Strings

Strings are sets of characters. Strings are easier to understand by looking at some examples.

Single and double quotes

Strings are contained by either single or double quotes.

```
my_string = "This is a double-quoted string." my_string = 'This is a single-quoted string
.'
```

This lets us make strings that contain quotations.

```
quote = "Linus Torvalds once said, 'Any program is only as good as it is useful.'"
```

Changing case

You can easily change the case of a string, to present it the way you want it to look.

```
first_name = 'eric'
print(first_name)
print(first_name.title())
```

Output:

```
eric Eric
```

It is often good to store data in lower case, and then change the case as you want to for presentation. This catches some TYpos. It also makes sure that 'eric', 'Eric', and 'ERIC' are not considered three different people.

Some of the most common cases are lower, title, and upper.

```
first_name = 'eric'
print(first_name)
print(first_name.title())
print(first_name.upper())

first_name = 'Eric'
print(first_name.lower())
```

Output:

```
eric Eric ERIC eric
```

You will see this syntax quite often, where a variable name is followed by a dot and then the name of an action, followed by a set of parentheses. The parentheses may be empty, or they may contain some values.

variable_name.action()

In this example, the word "action" is the name of a method. A method is something that can be done to a variable. The methods 'lower', 'title', and 'upper' are all functions that have been written into the Python language, which do something to strings. Later on, you will learn to write your own methods.

Combining strings (concatenation)

It is often very useful to be able to combine strings into a message or page element that we want to display. Again, this is easier to understand through an example.

```
first_name = 'ada' last_name = 'lovelace'  full_name = first_name + ' ' + last_name
print(full_name.title())
```
Output:
```
Ada Lovelace
```

The plus sign combines two strings into one, which is called "concatenation". You can use as many plus signs as you want in composing messages. In fact, many web pages are written as giant strings which are put together through a long series of string concatenations.

```
first_name = 'ada' last_name = 'lovelace' full_name = first_name + ' ' + last_name
message = full_name.title() + ' ' + "was considered the world's first computer programmer
."

print(message)
```
Output:
```
Ada Lovelace was considered the world's first computer programmer.
```

If you don't know who Ada Lovelace is, you might want to go read what Wikipedia or the Computer History Museum have to say about her. Her life and her work are also the inspiration for the Ada Initiative, which supports women who are involved in technical fields.

Whitespace

The term "whitespace" refers to characters that the computer is aware of, but are invisible to readers. The most common whitespace characters are spaces, tabs, and newlines.

Spaces are easy to create, because you have been using them as long as you have been using computers. Tabs and newlines are represented by special character combinations.

The two-character combination "\t" makes a tab appear in a string. Tabs can be used anywhere you like in a string.

```
print("Hello everyone!")
```
Output:
```
Hello everyone!
print("\tHello everyone!")
```
Output:
```
        Hello everyone!
print("Hello \teveryone!")
```

Output:
```
Hello   everyone!
```
The combination "\n" makes a newline appear in a string. You can use newlines anywhere you like in a string.

```
print("Hello everyone!")
```
Output:
```
Hello everyone!
```
```
print("\nHello everyone!")
```
Output:
```
Hello everyone!
```
```
print("Hello \neveryone!")
```
Output:
```
Hello  everyone!
```
```
print("\n\n\nHello everyone!")
```
Output:
```
  Hello everyone!
```

Stripping whitespace

Many times you will allow users to enter text into a box, and then you will read that text and use it. It is really easy for people to include extra whitespace at the beginning or end of their text. Whitespace includes spaces, tabs, and newlines.

It is often a good idea to strip this whitespace from strings before you start working with them. For example, you might want to let people log in, and you probably want to treat 'eric ' as 'eric' when you are trying to see if I exist on your system.

You can strip whitespace from the left side, the right side, or both sides of a string.

```
name = ' eric '
print(name.lstrip())
print(name.rstrip())
print(name.strip())
```
Output:
```
eric   eric eric
```
It's hard to see exactly what is happening, so maybe the following will make it a little more clear:

```
name = ' eric '
print('-' + name.lstrip() + '-')
print('-' + name.rstrip() + '-')
print('-' + name.strip() + '-')
```
Output:
```
-eric - - eric- -eric-
```

Exercises

Someone Said

- Find a quote that you like. Store the quote in a variable, with an appropriate introduction such as "Ken Thompson once said, 'One of my most productive days was throwing away 1000 lines of code'". **Print** the quote.

First Name Cases

- Store your first name, in lowercase, in a variable.
- Using that one variable, **print** your name in lowercase, Titlecase, and UPPERCASE.

Full Name

- Store your first name and last name in separate variables, and then combine them to **print** out your full name.

About This Person

- Choose a person you look up to. Store their first and last names in separate variables.
- Use concatenation to make a sentence about this person, and store that sentence in a variable.-
- **Print** the sentence.

Name Strip

- Store your first name in a variable, but include at least two kinds of whitespace on each side of your name.
- **Print** your name as it is stored.
- **Print** your name with whitespace stripped from the left side, then from the right side, then from both sides.

Numbers

Dealing with simple numerical data is fairly straightforward in Python, but there are a few things you should know about.

Integers

You can do all of the basic operations with integers, and everything should behave as you expect. Addition and subtraction use the standard plus and minus symbols. Multiplication uses the asterisk, and division uses a forward slash. Exponents use two asterisks.

```
print(3+2)
```
Output:
```
5
```
```
print(3-2)
```
Output:
```
1
```
```
print(3*2)
```
Output:
```
6
```
```
print(3/2)
```
Output:
```
1.5
```
```
print(3**2)
```
Output:
```
9
```

You can use parenthesis to modify the standard order of operations.

```
standard_order = 2+3*4
print(standard_order)
```
Output:
```
14
```
```
my_order = (2+3)*4
print(my_order)
```
Output:
```
20
```

Floating-Point numbers

Floating-point numbers refer to any number with a decimal point. Most of the time, you can think of floating point numbers as decimals, and they will behave as you expect them to.

```
print(0.1+0.1)
```
Output:
```
0.2
```

However, sometimes you will get an answer with an unexpectly long decimal part:

```
print(0.1+0.2)
```
Output:
```
0.30000000000000004
```

This happens because of the way computers represent numbers internally; this has nothing to do with Python itself. Basically, we are used to working in powers of ten, where one tenth plus two tenths is just three tenths. But computers work in powers of two. So your computer has to represent 0.1 in a power of two, and then 0.2 as a power of two, and express their sum as a power of two. There is no exact representation for 0.3 in powers of two, and we see that in the answer to 0.1+0.2.

Python tries to hide this kind of stuff when possible. Don't worry about it much for now; just don't be surprised by it, and know that we will learn to clean up our results a little later on.

You can also get the same kind of result with other operations.

```
print(3*0.1)
```
Output:
```
0.30000000000000004
```

Exercises

Arithmetic

- Write a program that **print**s out the results of at least one calculation for each of the basic operations: addition, subtraction, multiplication, division, and exponents.

Order of Operations

- Find a calculation whose result depends on the order of operations.
- **Print** the result of this calculation using the standard order of operations.
- Use parentheses to force a nonstandard order of operations. **Print** the result of this calculation.

Long Decimals

- On paper, 0.1+0.2=0.3. But you have seen that in Python, 0.1+0.2=0.30000000000000004.
- Find at least one other calculation that results in a long decimal like this.

Challenges

Neat Arithmetic

- Store the results of at least 5 different calculations in separate variables. Make sure you use each operation at least once.
- **Print** a series of informative statements, such as "The result of the calculation 5+7 is 12."

Neat Order of Operations

- Take your work for "Order of Operations" above.
- Instead of just **print**ing the results, **print** an informative summary of the results. Show each calculation that is being done and the result of that calculation. Explain how you modified the result using parentheses.

Long Decimals - Pattern

- On paper, 0.1+0.2=0.3. But you have seen that in Python, 0.1+0.2=0.30000000000000004.
- Find a number of other calculations that result in a long decimal like this. Try to find a pattern in what kinds of numbers will result in long decimals

Comments

As you begin to write more complicated code, you will have to spend more time thinking about how to code solutions to the problems you want to solve. Once you come up with an idea, you will spend a fair amount of time troubleshooting your code, and revising your overall approach.

Comments allow you to write in English, within your program. In Python, any line that starts with a pound (#) symbol is ignored by the Python interpreter.

```
# This line is a comment. print("This line is not a comment, it is code.")
```

Output:

```
This line is not a comment, it is code.
```

What makes a good comment?

- It is short and to the point, but a complete thought. Most comments should be written in complete sentences.
- It explains your thinking, so that when you return to the code later you will understand how you were approaching the problem.
- It explains your thinking, so that others who work with your code will understand your overall approach to a problem.
- It explains particularly difficult sections of code in detail.

When should you write comments?

- When you have to think about code before writing it.
- When you are likely to forget later exactly how you were approaching a problem.
- When there is more than one way to solve a problem.
- When others are unlikely to anticipate your way of thinking about a problem.

Writing good comments is one of the clear signs of a good programmer. If you have any real interest in taking programming seriously, start using comments now. You will see them throughout the examples in these notebooks.

Exercises

First Comments

- Choose the longest, most difficult, or most interesting program you have written so far. Write at least one comment in your program.

Overall Challenges

We have learned quite a bit so far about programming, but we haven't learned enough yet for you to go create something. In the next notebook, things will get much more interesting, and there will be a longer list of overall challenges.

What I've Learned

- Write a program that uses everything you have learned in this notebook at least once.
- Write comments that label each section of your program.
- For each thing your program does, write at least one line of output that explains what your program did.
- For example, you might have one line that stores your name with some whitespace in a variable, and a second line that strips that whitespace from your name:

```
# I learned how to strip whitespace from strings. name = '\t\teric' print("I can strip tabs from my name: " + name.strip())
```

Output:

```
I can strip tabs from my name: eric
```

Motivate to Innovate...

STEM BUILDERS Learning Center

MATH | ROBOTICS | TECHNOLOGY | EVENTS | CAMPS

contactus@stembuilders.com

www.stembuilders.com